05 · Firmware

Doc: LOT-HW-01-FW · **Rev:** A (2026-11-14) · **Source:** `../firmware/`

1. STACK

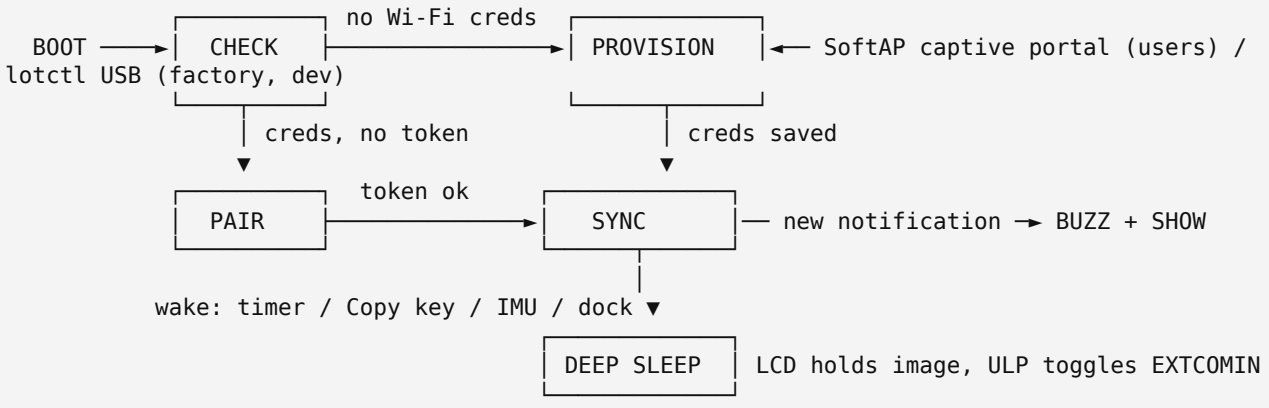
- **Target:** ESP32-S3 (EVT: ESP32-S3-MINI-1-N4R2, dev: ESP32-S3-DevKitC-1-N8R8)
- **Framework:** Arduino-ESP32 3.x on ESP-IDF 5.x, built with PlatformIO (`firmware/platformio.ini`)
- **Libraries:** `esp32-camera`, Adafruit SHARP Memory Display + GFX, Bosch BSEC2 (BME688), SparkFun BMI270, Adafruit DRV2605, ArduinoJson 7, SparkFun MAX1704x

2. MODULE LAYOUT

```
firmware/
├── platformio.ini
├── include/
│   ├── config.h        pins, cadence, API host, limits
│   └── lot_api.h       API client interface
└── src/
    ├── main.cpp        boot → state machine → deep sleep
    ├── lot_api.cpp     HTTPS client: register, pair, sync, copy, photo, telemetry
    ├── ui.cpp          Memory LCD rendering (LOT typography, period style)
    ├── haptic.cpp      DRV2605L "pager" pattern
    ├── sensors.cpp     BME688/BSEC2, LTR-303, fuel gauge
    ├── camera.cpp      single-frame JPEG capture with power gating
    └── provision.cpp   USB-serial line protocol for lotctl
```

Session 1 ships `main.cpp`, `config.h`, `lot_api.h/.cpp` and the provisioning protocol. The other modules are stubbed inside `main.cpp` with TODO markers.

3. STATE MACHINE




```
 Copy short press: POST /copy for the currently shown nid → "Copied."  (no notification:
"Nothing to copy.")
 Copy long press (≥ 800 ms): camera frame → POST /photo → "Copied. Photo."
 Copy hold 10 s: forget token → PAIR
```

Provisioning paths. End users have no USB port (the Tag-Connect pads are internal), so v1 adds a SoftAP captive portal: the AP is named `LOT-TILE-<last 4 of serial>`, the page is served at `192.168.4.1`, and it times out after 10 minutes. `lotctl` over USB-Serial/JTAG is for factory test and development. BLE provisioning

through Web Bluetooth on lot-systems.com is a v2 option (it is not supported in iOS Safari). *Session 1 firmware implements the `lotctl` path only. The SoftAP portal is a phase-2 task.*

4. WAKE SOURCES

Source	ESP32-S3 mechanism	Action
Timer	`esp_sleep_enable_timer_wakeup(next_poll_s)`	SYNC. Telemetry every 3rd wake
Copy key (GPIO0)	`ext1` on RTC GPIO, active low	Copy / photo / re-pair
IMU pickup (GPIO1)	`ext1`	Refresh screen with time and weather, quick SYNC
Dock (PGOOD)	`ext1`	Show "Charging." Allow OTA check while on power

5. RENDERING RULES (LOT STYLE)

- One line of notification text, wrapped to at most 2 lines at 160 px. A second, tertiary-weight line shows `HH:MM · 12°C`.
- No icons and no emoji. Status words end with a period: `Copied.` `Charging.` `Offline.` `Paired.`
- The previous notification stays on screen (Memory LCD), so the device reads as "on" with the MCU off.

6. OTA

Two OTA app partitions plus `nvs` and `otadata`. OTA runs only on the dock or at ≥ 50 % battery. Firmware calls `POST /api/device/ota/check`, streams the image, verifies SHA-256 and the ECDSA signature, sets the boot partition, and reboots. If the new image does not reach `SYNC` within 2 boots, the bootloader rolls back (`CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE`).

7. FACTORY PROGRAMMING AND TEST

1. Bed-of-nails jig with Tag-Connect TC2030 to USB-Serial/JTAG.
2. `esptool.py` flashes the test firmware. The test runs: I²C scan (expect 0x76/0x77 BME688, 0x68/0x69 BMI270, 0x29 LTR-303, 0x5A DRV2605, 0x36 MAX17048), LCD checkerboard, a camera frame-hash check, haptic current draw, Qi charge detect on a pad, and Wi-Fi RSSI against a reference AP.
3. Burn the serial into eFuse user block (`LT01-000001`…`LT01-000100`), then flash production firmware with flash encryption.

8. HOST SOFTWARE: `LOTCTL` (REQUIREMENT 10)

`software/lotctl.py` talks to the firmware over the USB-serial line protocol:

```
> HELLO                        < OK LOT-TILE LT01-000042 fw=0.1.0 hw=EVT1
> WIFI <ssid> <pass>           < OK saved
> API <https://lot-systems.com>   < OK
> STATUS                       < OK wifi=up rssi=-58 paired=yes bat=81 nid=…
> PAIR                         < OK code=K7Q-2MX
> FORGET                       < OK
> LOG                          < (streams firmware log lines until Ctrl-C)
```

`lotctl simulate` runs the same API flow as the firmware, from a laptop, with no hardware. Use it to build and test the server routes before EVT boards exist.
